

AI-Powered Audit Document Tracking and Follow- up

Week 10: Workflow Automation Capstone

Submitted by: Leilah Kokoyo & Dennis Paul Date: June 2026

Dennis Paul and Associates never misses an overdue audit document again.

We engineered a production-grade AI workflow that classifies risk, drafts professional emails, gates them through human approval, and logs every decision automatically.

THE PROBLEM

Let's talk about losing audit efficiency in plain sight.

Dennis Paul and Associates processes document requests for ten or more clients at any given time. Mixed into those daily requests are the ones that matter most: overdue submissions that directly threaten audit timelines. But because all pending items lived in the same Google Sheet, overdue documents were going unnoticed. By the time any team member spotted a critically delayed submission, the window for a firm but professional follow-up had already passed.

The previous approach to fixing this with automation kept failing. The culprit: manual processes. Follow-up emails were hardcoded templates the same flat text sent regardless of whether a client was three days late or twenty-five days late. There was no risk classification, no intelligent drafting, and no oversight layer before emails went out to real clients.

The real problem was not just sending reminder emails. It was building a system robust enough to classify risk intelligently, generate tone-appropriate communication, require a human decision before anything reached a client, and log every outcome all without human intervention in the daily execution.

SOLUTION

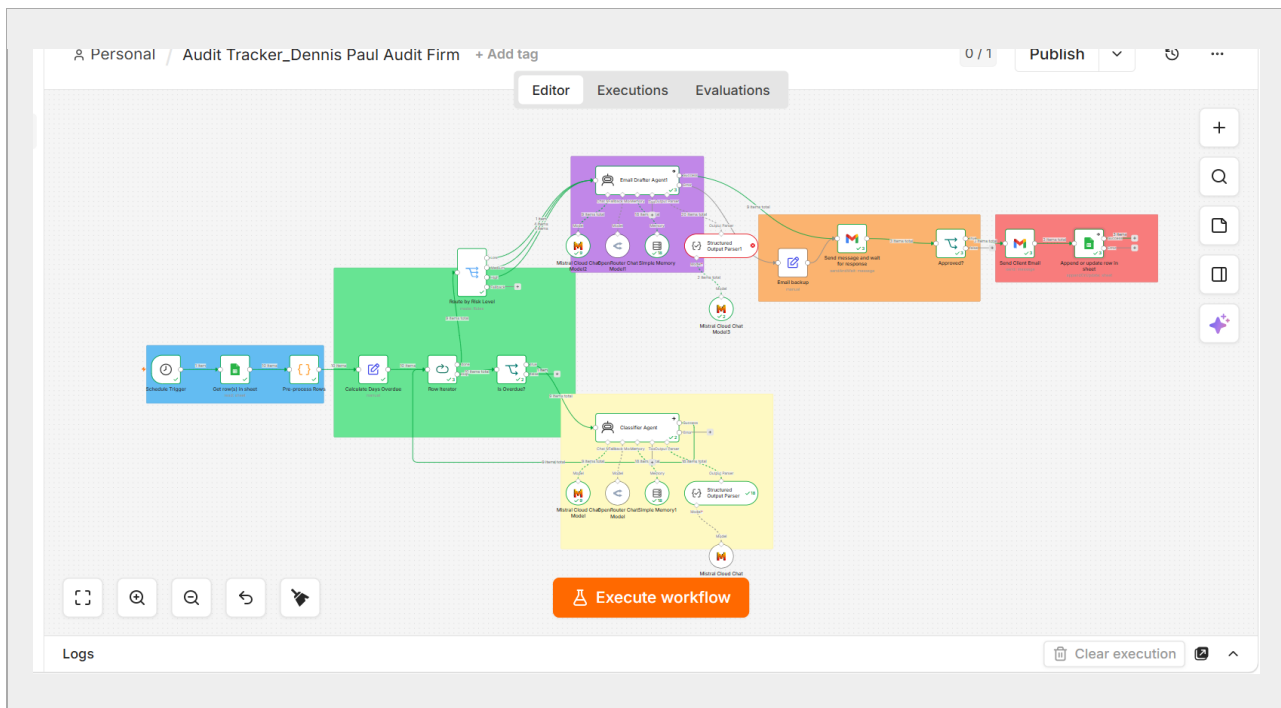
We built the AI-Powered Audit Document Tracking and follow up System workflow in n8n. Every morning at 8:00 AM, the workflow fires automatically. It reads the Audit Tracker Google Sheet, calculates how many days each document is overdue, filters out non-overdue records, runs each overdue item through an AI Classifier Agent for risk scoring, generates a tone-matched reminder email via a second AI Drafter Agent, routes Medium and High risk emails through a manager approval gate, sends approved emails to clients, and logs every outcome to both Google Sheets and Airtable all without human intervention in execution.

Five engineering decisions make this production-grade:

- A Pre-process Rows Code node normalizes date formats and assigns unique row IDs before any calculation runs.

- An Is Overdue? IF node filters out records that have not yet passed their due date, preventing unnecessary AI calls.
- Two dedicated AI Agents with separate responsibilities: the Classifier Agent scores risk only; the Email Drafter Agent writes emails only.
- A human approval gate using n8n's Wait node pauses execution and emails the manager with Approve and Reject links before any client email is sent.
- A separate AI-powered Error Handler workflow catches any failures and emails an intelligent diagnosis to the team.

Figure 1 — Full n8n Canvas — Six colour-coded visual groups from left to right: Data Input (Blue), Loop & Logic (Green), Classifier AI (yellow), Email Drafter AI (Purple), Approval Flow (orange), Output (Red).



STEP 1: DATA INPUT & PRE-PROCESSING

Schedule Trigger + Get row(s) in sheet + Pre-process Rows

The workflow starts with a Schedule Trigger configured to fire every day at 8:00 AM. No manual run. No human needed to start the process.

Immediately after the trigger, a Google Sheets node named Get row(s) in sheet reads all rows from the Audit Tracker sheet where Status equals Pending. The sheet contains the following columns:

Column	Type	Description
Client ID	String	Unique identifier for each client (e.g. DPA001)
Client Name	String	Name of the audit client
Client Email	String	Email address for reminder delivery
Document Requested	String	Name of the outstanding document
Due Date	Date	Submission deadline for the document
Status	String	Current status: Pending or Submitted
Days Pending	Number	Calculated overdue days (written back by workflow)
Risk Level	String	AI-assigned classification: Low, Medium, or High
Reminder Sent	String	Flag updated to YES after email is sent

After reading the rows, a Code node named Pre-process Rows normalizes the data before it enters any calculation. The node adds two fields to every row:

```
return items.map((item, i) => ({
  ...item,
  json: {
    ...item.json,
    row_id: i + 1,
    normalized_due: new Date(item.json['Due Date']).toISOString()
  }
}));
```

The design principle here is deliberate. Date happens once, at the source, in a dedicated node. Downstream calculations never need to guess whether they are receiving a raw Google Sheets date string or an ISO timestamp. The row_id field ensures each record is uniquely trackable through the rest of the flow.

Figure 2 — Schedule Trigger NODE — Configured to fire at 08:00 every day. One signal initiates the full audit processing run.

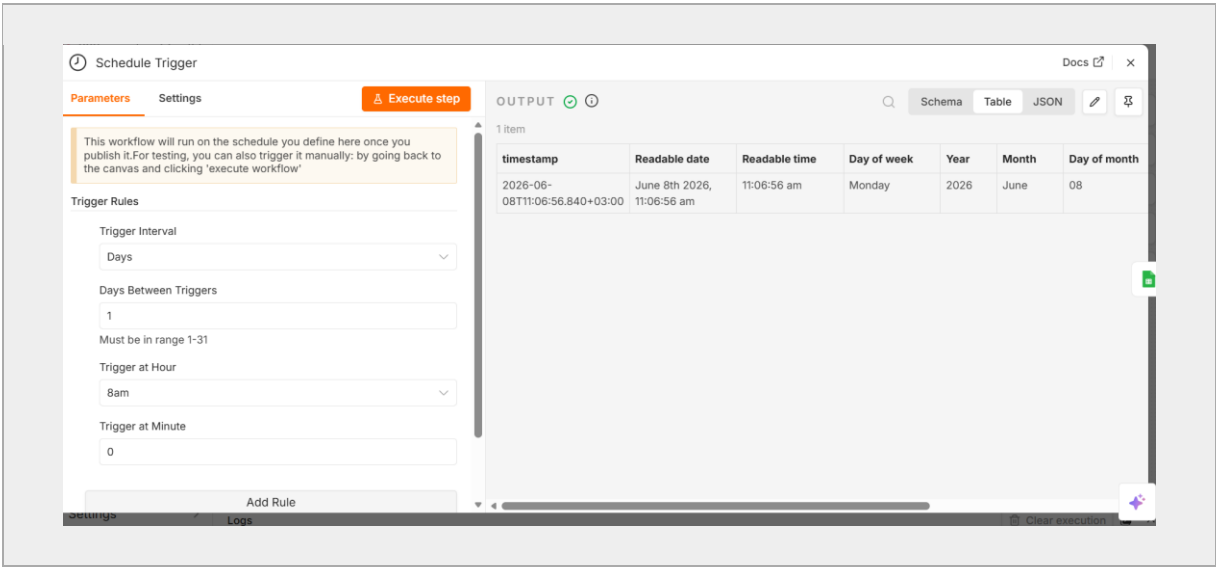


Figure 3 — Pre-process Rows CODE NODE —Due Date to ISO format and assigns a row_id to each record before downstream calculation.

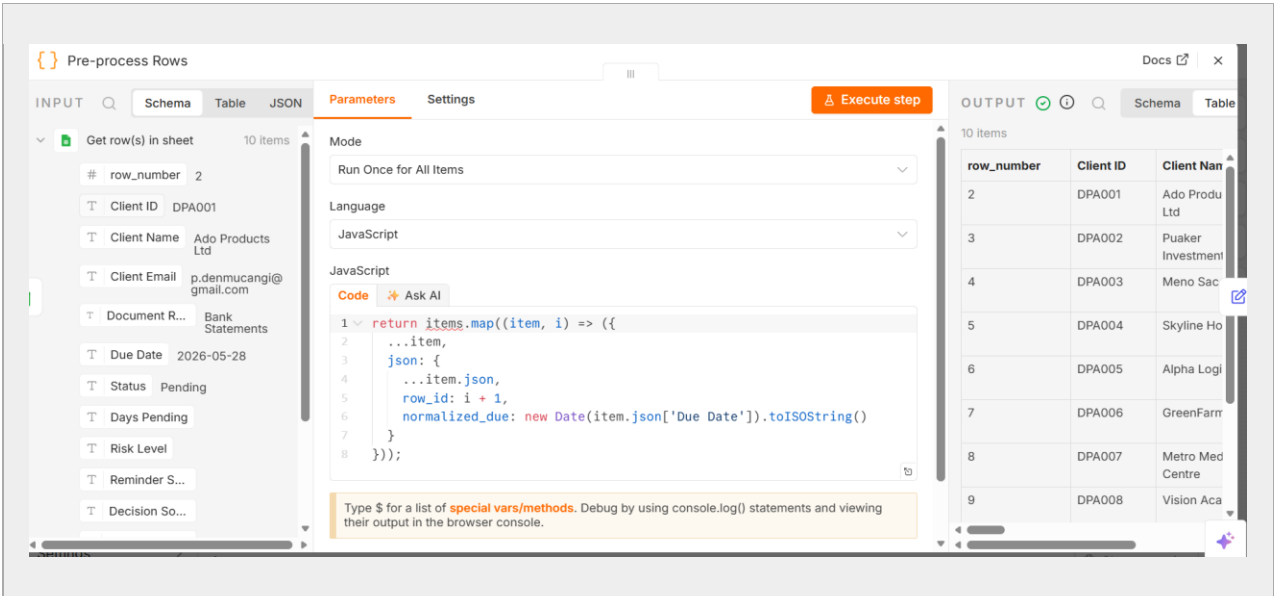


Figure 4 — Data Input Visual Group (BLUE) — Schedule Trigger, Google Sheets, and Pre-process Rows nodes grouped and colour-coded blue.

Get row(s) in sheet

INPUT

Schedule Trigger

T timestamp

2026-06-08T11:06:56.840+03:00

T Readable date

June 8th 2026, 11:06:56 am

T Readable time

11:06:56 am

T Day of week

Monday

T Year

2026

T Month

June

T Day of month

08

T Hour

11

T Minute

06

T Second

56

T Timezone

Africa/Nairobi (UTC+03:00)

Variables and context

Parameters

Credential

Google Sheets account

Resource

Sheet Within Document

Operation

Get Row(s)

Document

By URL

https://docs.google.com/spreadsheets/d/

Sheet

From list

Audit tracker

Filters

Currently no items exist

Add Filter

Settings

Execute step

OUTPUT

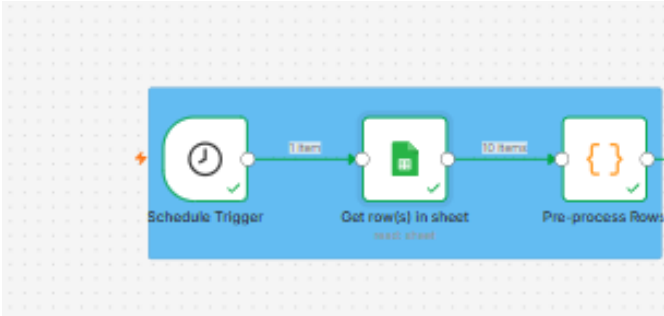
Schema

Table

JSON

10 Items

row_number	Client ID	Client Name	Client E
2	DPA001	Ado Products Ltd	p.denm
3	DPA002	Puaker Investment Co.	dpassor
4	DPA003	Meno Sacco	curtisbr
5	DPA004	Skyline Holdings	leilahkc
6	DPA005	Alpha Logistics	kellykey
7	DPA006	GreenFarm Ltd	leilah.kc
8	DPA007	Metro Medical Centre	dennis.i
9	DPA008	Vision Academy	info@dt
10	DPA009	Elite Hardware	dpassor
11	DPA010	Sunlight Wholesalers	jeptimkc



STEP 2: LOOP & OVERDUE FILTERING

Row Iterator + Calculate Days Overdue + Is Overdue?

After pre-processing, the data enters a Loop Over Items node named Row Iterator with a batch size of 1. This is the most important structural decision in the workflow. Because the AI Classifier Agent must evaluate one client at a time, the loop ensures it never receives a batch of rows. Each row travels independently through the entire downstream pipeline — classified, drafted, approved, and logged individually — before the loop picks up the next record.

From the loop output, each row reaches the Calculate Days Overdue Set node. This node computes three fields using the normalised due date:

Field	Type	Expression
Days Pending	Number	Math.floor((Date.now() - new Date(\$json['Due Date']).getTime()) / 86400000)
overdue	String	Returns 'true' if Days Pending > 0, 'false' otherwise
days_label	String	Human-readable label: '7 days overdue', 'Due today', '3 days remaining'

The Days Pending field is typed as Number explicitly. This prevents the type mismatch error that caused the original workflow to fail when the IF node attempted a numeric comparison against a string value.

After calculation, each row passes through an IF node named Is Overdue?. This is the Scope Guard for the AI pipeline. The condition is simple:

```
Condition → {{ $json['Days Pending'] }} is greater than 0    (type: Number)
```

Rows where Days Pending is zero or negative exit on the False branch and are returned to the Row Iterator without any AI processing. They were valid data but not overdue, so no action is needed. This guard eliminates unnecessary AI API calls and prevents the Classifier Agent from ever receiving an irrelevant row.

The Row Iterator, Calculate Days Overdue, and Is Overdue? nodes are grouped into a Visual Group coloured GREEN, labelled Loop & Logic. This immediately signals to any developer opening the canvas that this is where row-level iteration and the overdue filter live.

Figure 5 — Row Iterator NODE — Loop Over Items node with batch size 1, ensuring the AI Classifier processes exactly one client record per cycle.

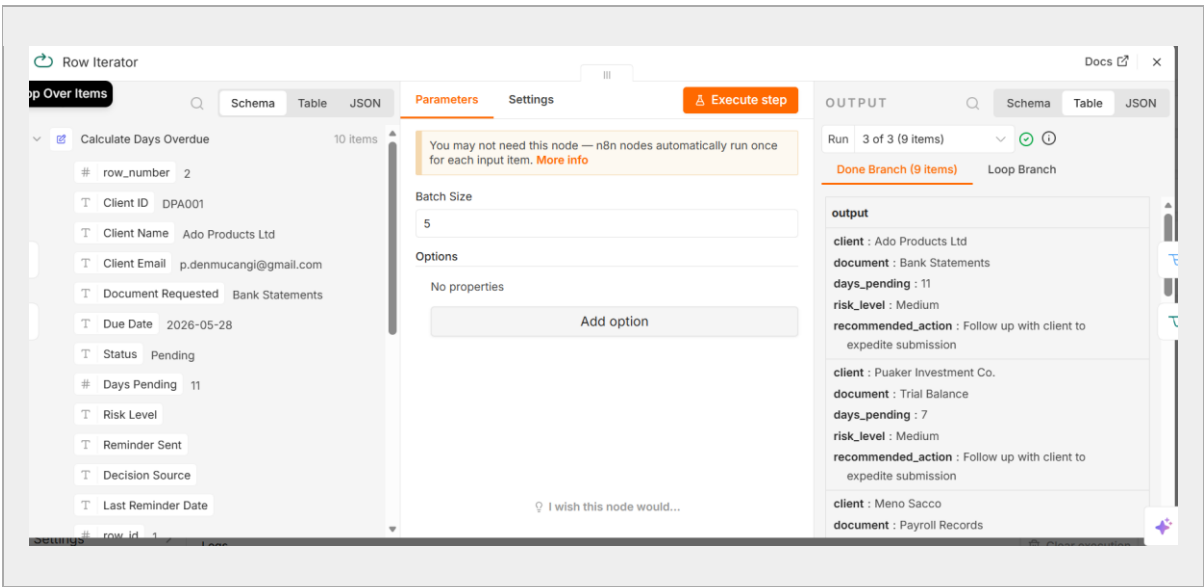


Figure 6 — Is Overdue? NODE — IF node filtering non-overdue records before they reach the AI pipeline. False branch returns to the Row Iterator.

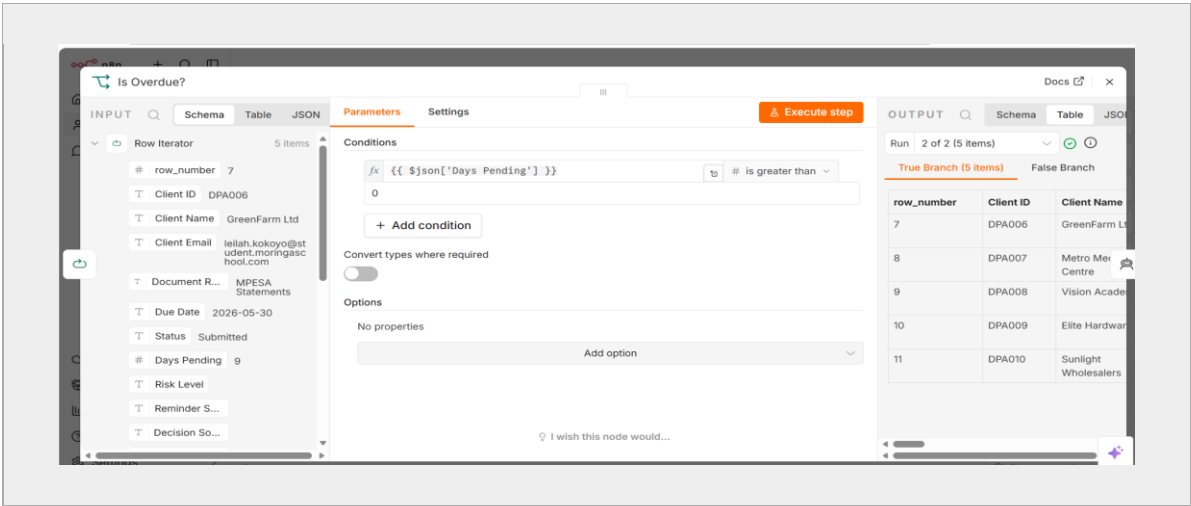
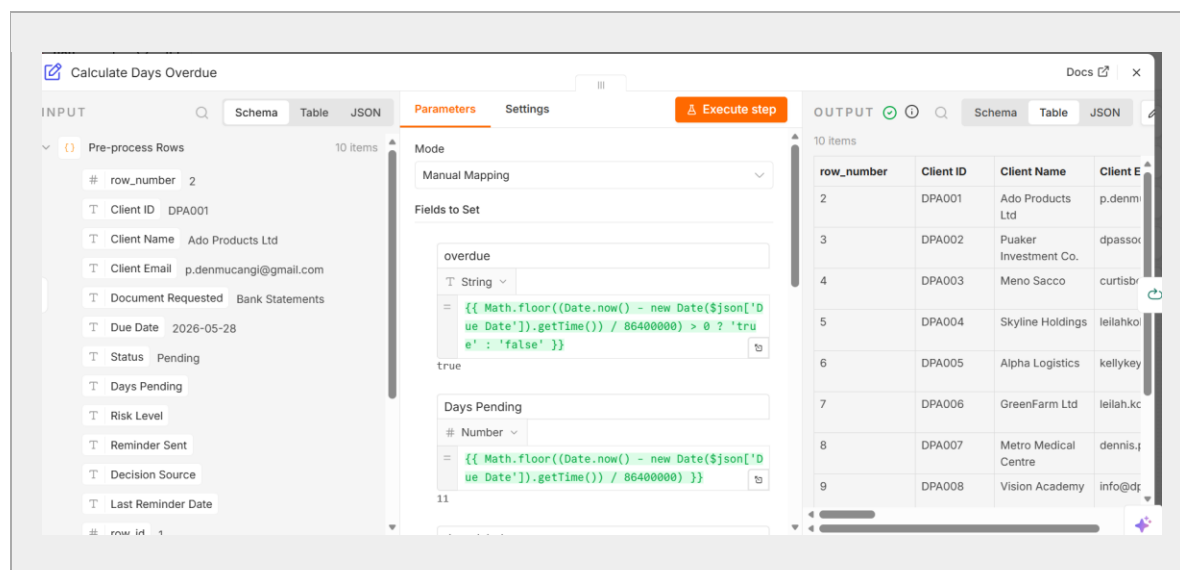


Figure 7 — Loop & Logic Visual Group (GREEN) — Row Iterator, Calculate Days Overdue, and Is Overdue? nodes grouped and colour-coded green.



STEP 3: INTELLIGENCE LAYER — AI CLASSIFIER AGENT

Classifier Agent + Route by Risk Level + Write to Classifications Tab

Every overdue row that clears the Is Overdue? gate reaches the first intelligence layer: an AI Agent node named Classifier Agent. This agent has one responsibility only — assess the risk level of the delay and return a structured classification. It does not draft emails. It does not make decisions about tone. Its output is a single JSON object.

The agent is backed by a Mistral Cloud Chat Model with temperature set to 0.2 for focused, consistent output. A Structured Output Parser enforces the JSON schema. Auto-Fix is enabled so minor formatting variations from the model are corrected before the data moves downstream.

The Classifier Agent node connects to three sub-nodes:

- Mistral Cloud Chat Model — primary language model for risk classification.
- Simple Memory — session memory keyed by timestamp, preventing context bleed between client records.
- Structured Output Parser — enforces the seven-field JSON schema shown above.

After classification, the output fans to two parallel destinations simultaneously:

- Write to Classifications Tab — a Google Sheets Append Row node that logs the classification result to a dedicated Classifications sheet tab, recording Client Name, Document, Days Pending, Risk Level, Recommended Action, Tone, Requires Escalation, and Processed At.
- Route by Risk Level — a Switch node that routes the classified record to the appropriate downstream branch based on risk_level.

The Switch node contains three rules, each referencing the AI output directly:

```
Rule 1 → {{ $json.output.risk_level }} equals Low → Output 0
Rule 2 → {{ $json.output.risk_level }} equals Medium → Output 1
Rule 3 → {{ $json.output.risk_level }} equals High → Output 2
```


The critical design discipline: the values Low, Medium, and High in the Switch conditions must match the AI output exactly, character for character. The AI system message explicitly constrains risk_level to one of these three strings. The Switch node does not contain the word 'Risk' — because the AI does not return it.

Figure 8 — Classifier Agent NODE — AI Agent classifying each overdue record. System prompt constrains output to exactly one of Low, Medium, or High with full JSON schema.

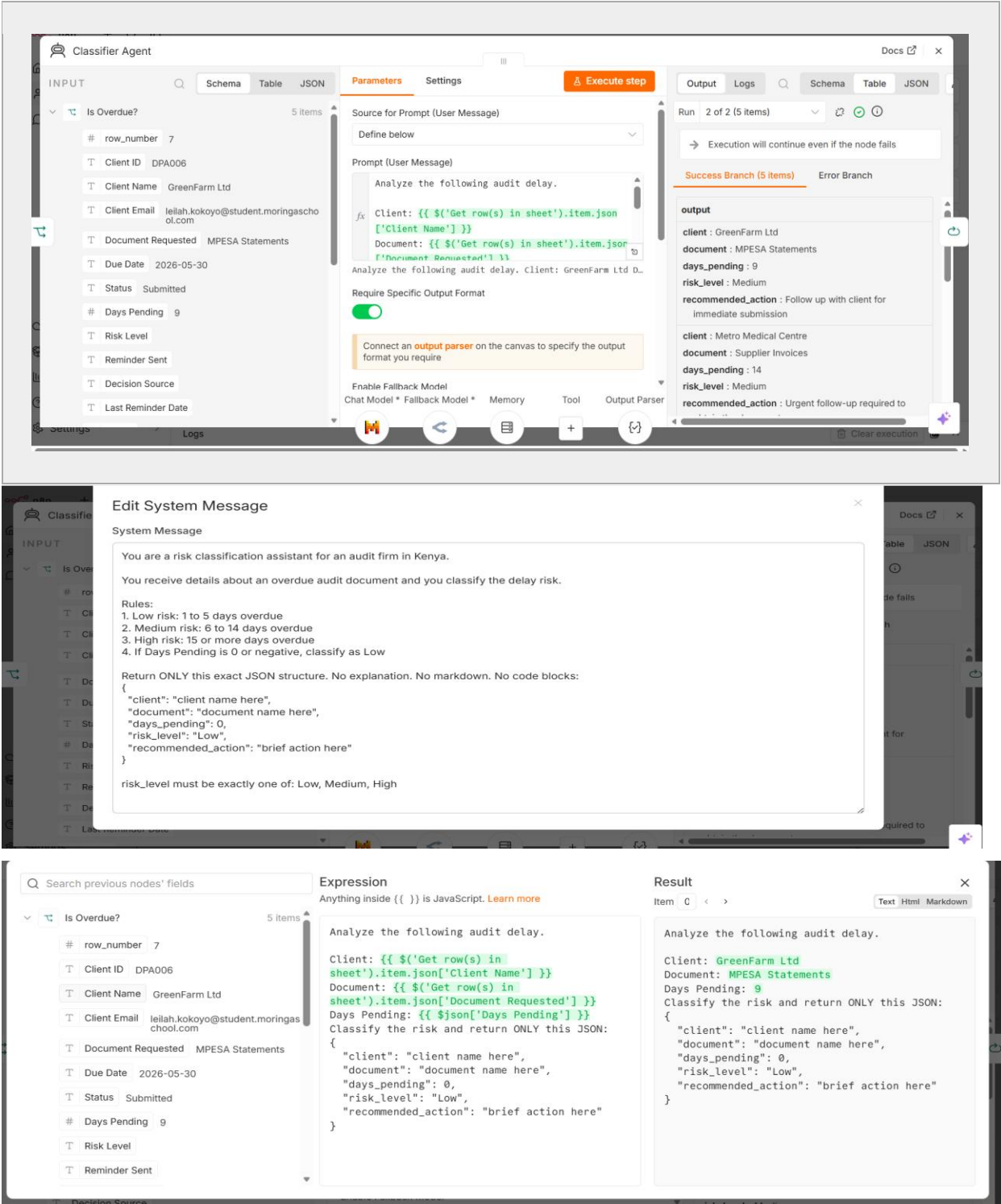


Figure 9 — Structured Output Parser NODE — Enforces the seven-field JSON schema. Auto-Fix enabled to handle minor AI formatting variations.

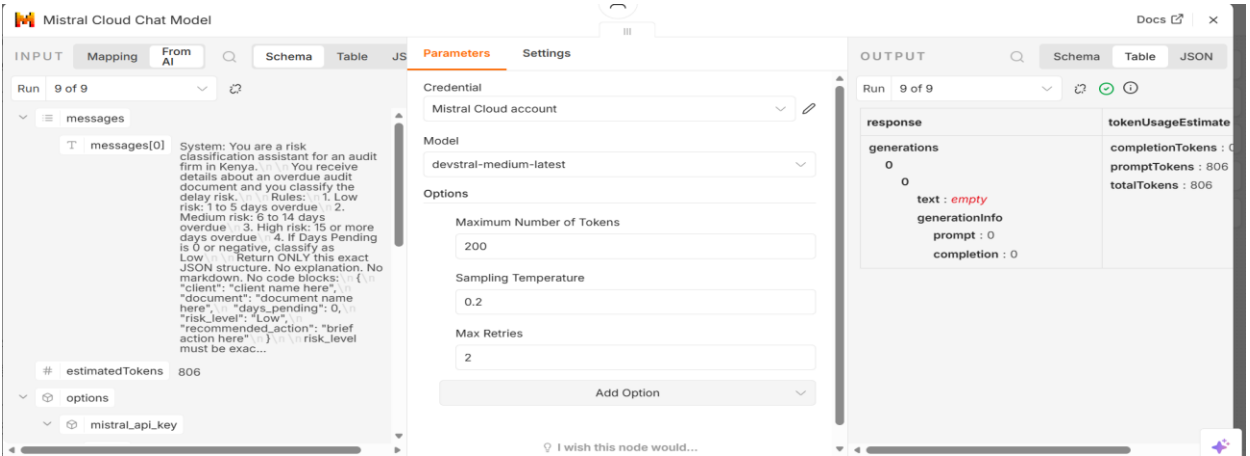
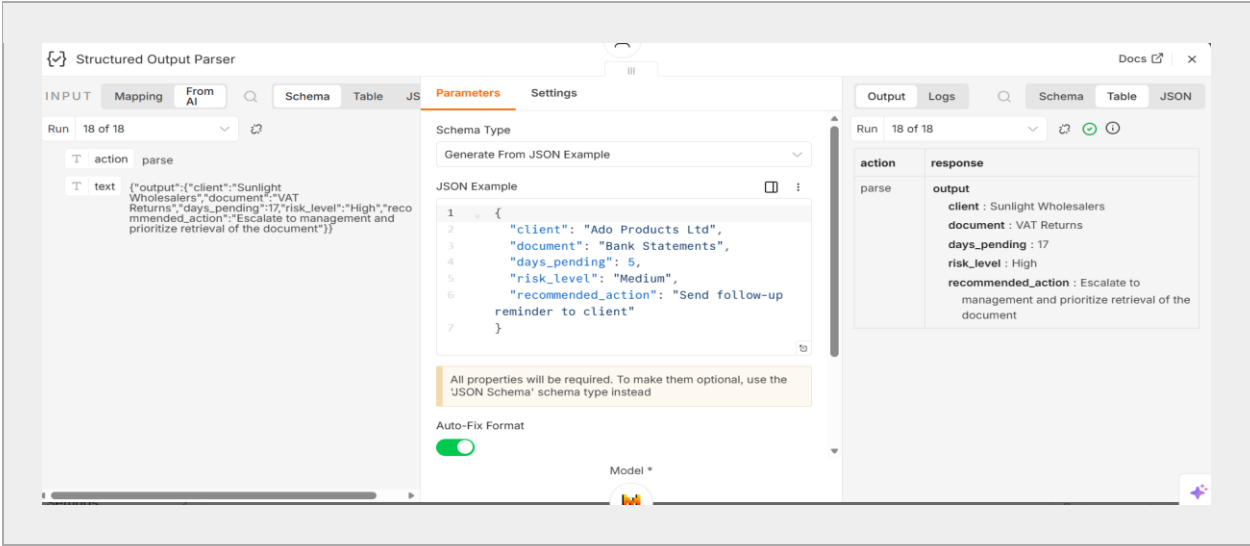


Figure 10 — Route by Risk Level NODE — Switch node routing Low, Medium, and High risk records to the appropriate email drafting branch.

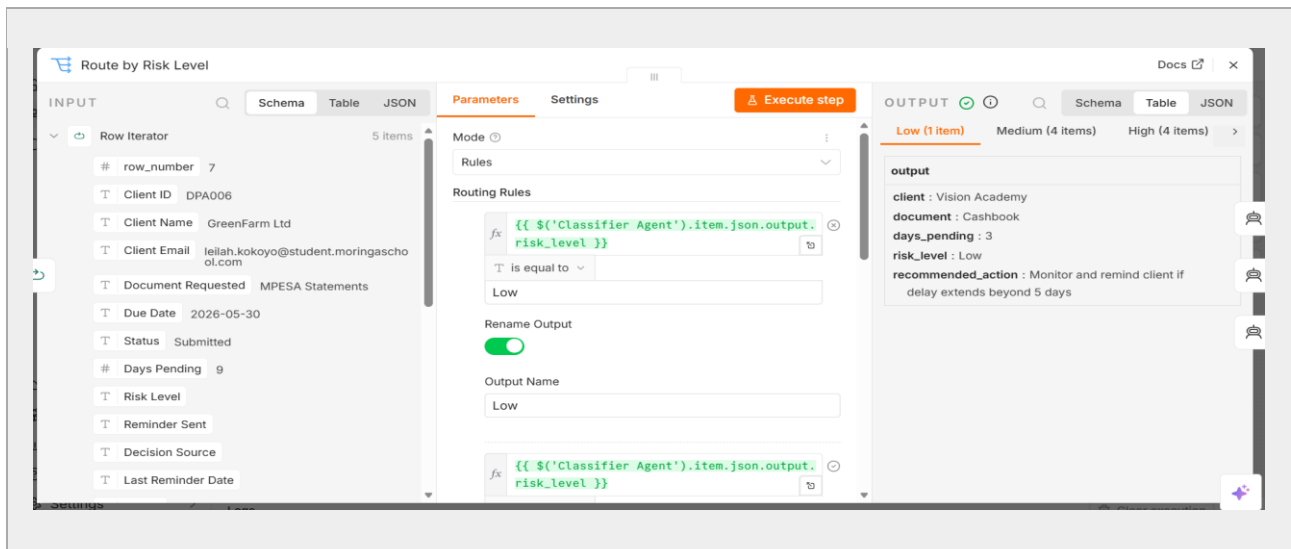
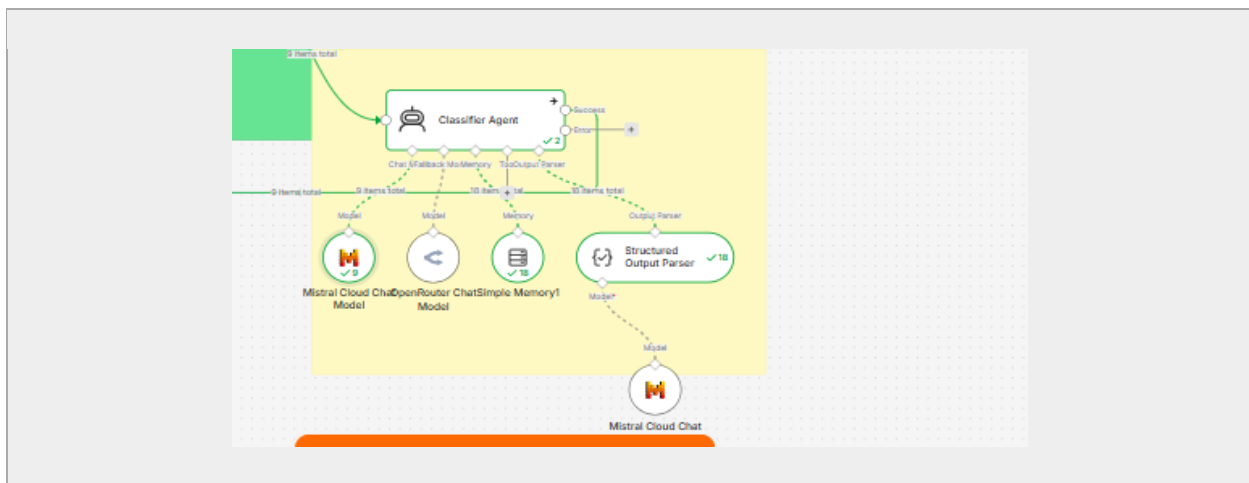


Figure 11 — Classifier AI Visual Group (Yellow) — Classifier Agent, Route by Risk Level, and Write to Classifications Tab nodes grouped and colour-coded amber.



STEP 4: INTELLIGENCE LAYER — EMAIL DRAFTER AGENT

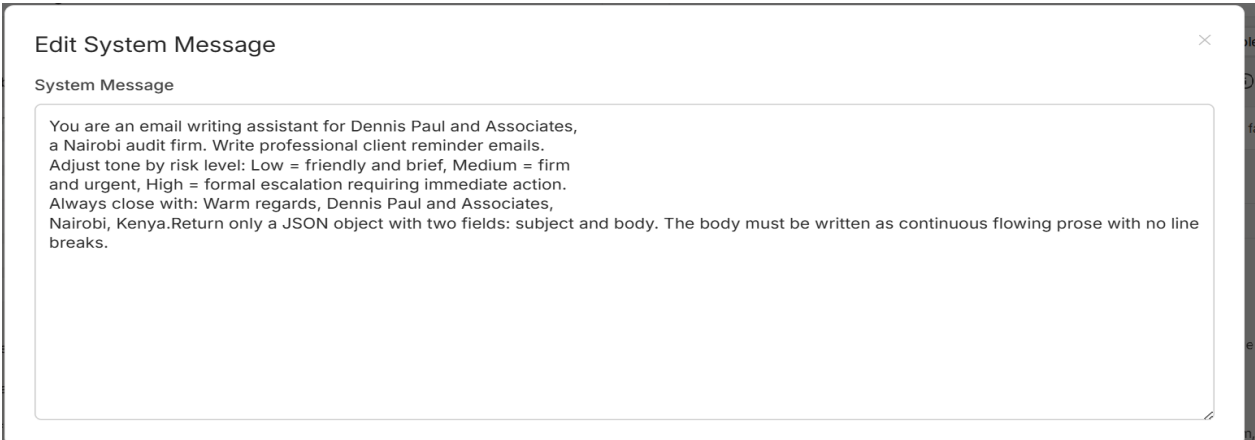
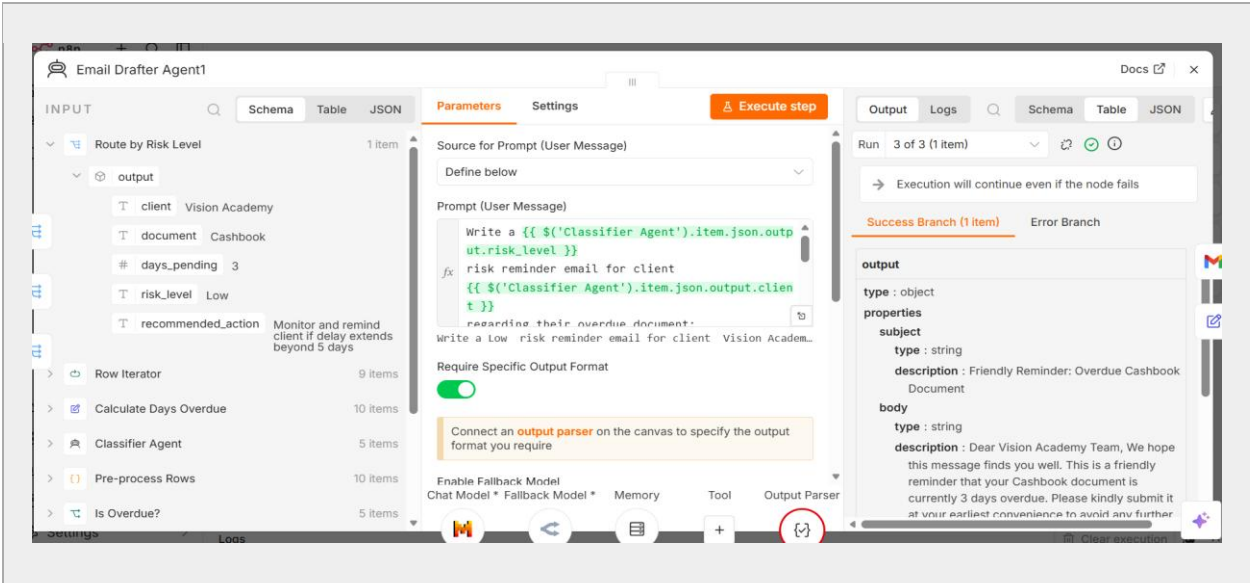
Email Drafter Agent

Every record that clears the Route by Risk Level Switch — regardless of which branch it takes — reaches the second intelligence layer: an AI Agent node named Email Drafter Agent. This agent has one responsibility only — write the client-facing reminder email. It does not re-classify risk. It uses the classification produced by the Classifier Agent to determine tone.

All three Switch branches (Low, Medium, High) feed into the same Email Drafter Agent node. The separation of concerns between the two agents is intentional. The Classifier Agent makes the analytical decision. The Email Drafter Agent executes the communication. If either agent needs to be updated independently, there is no risk of breaking the other.

The output is accessed downstream as `$json.output.email_body`. This clean reference replaces all three hardcoded Gmail template bodies from the original workflow. One AI agent, writing dynamically, replaces three static templates.

Figure 12 — Email Drafter Agent NODE — Second AI agent drafting tone-matched client emails. All three Switch branches feed into this single node.



The screenshot displays the 'Email Drafter AI Visual Group (PURPLE)' interface. It features a workflow editor on the left with nodes like 'Route by Risk Level', 'Row Iterator', 'Calculate Days Overdue', and 'Classifier Agent'. The 'Expression' pane shows JavaScript code for generating an email. The 'Result' pane shows the output of the expression, which is a risk reminder email for client 'GreenFarm Ltd' regarding an overdue document. Below this, the 'Structured Output Parser1' is shown, which takes the email text and outputs a structured JSON object with fields like 'subject', 'body', and 'recommended_action'.

Expression

```
Write a {{ $('Classifier Agent').item.json.output.risk_level }} risk reminder email for client {{ $('Classifier Agent').item.json.output.client }} regarding their overdue document: {{ $('Classifier Agent').item.json.output.document }}. It is {{ $('Classifier Agent').item.json.output.days_pending }} days overdue. Recommended action: {{ $('Classifier Agent').item.json.output.recommended_action }}. Write the email as continuous flowing paragraphs. Do not add a line break after every word or phrase.
```

Result

```
Write a Medium risk reminder email for client GreenFarm Ltd regarding their overdue document: MPESA Statements. It is 9 days overdue. Recommended action: Follow up with client for immediate submission. Write the email as continuous flowing paragraphs. Do not add a line break after every word or phrase.
```

Structured Output Parser1

INPUT Mapping From AI Schema Table JS Parameters Settings

Run 20 of 20

T action parse

T text `{"output":{"type":"object","properties":{"subject":{"type":"string","description":"Friendly Reminder: Overdue Cashbook Document"},"body":{"type":"string","description":"Dear Vision Academy Team, We hope this message finds you well. This is a friendly reminder that your Cashbook document is currently 3 days overdue. Please kindly submit it at your earliest convenience to avoid any further delays. If you need any assistance or have any questions, feel free to reach out to us. Thank you for your prompt attention to this matter. Warm regards, Dennis Paul and Associates, Nairobi, Kenya."},"require..."`

Schema Type: Generate From JSON Example

JSON Example

```
1 {
2   "type": "object",
3   "properties": {
4     "subject": {
5       "type": "string",
6       "description": "The email subject line"
7     },
8     "body": {
9       "type": "string",
10      "description": "The full email body as one continuous flowing"
11    }
12  }
13 }
```

All properties will be required. To make them optional, use the 'JSON Schema' schema type instead

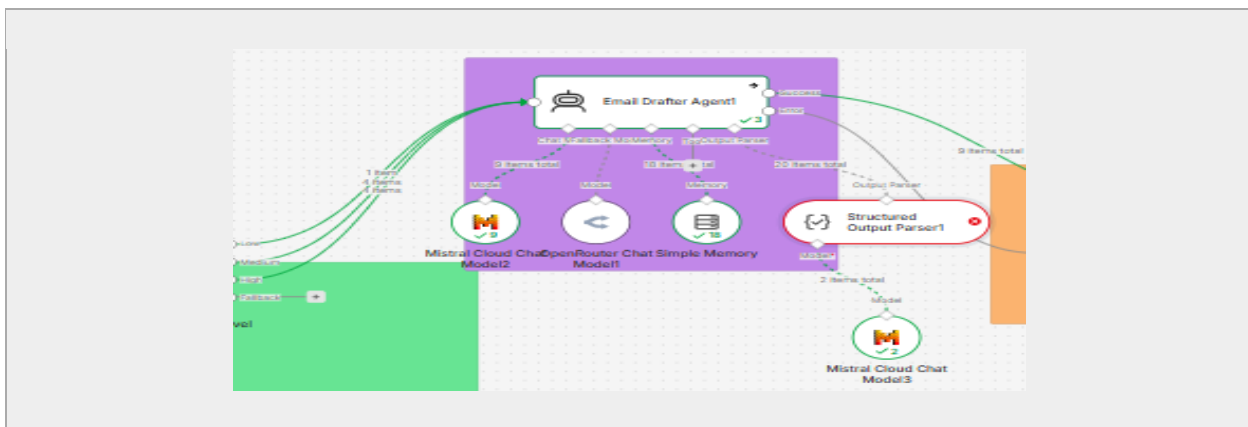
Model *

Output Logs Schema Table JSON

Run 20 of 20

action	response
parse	<pre>output: { type: object properties: { subject: { type: string description: Friendly Reminder: Overdue Cashbook Document } body: { type: string description: Dear Vision Academy Team, We hope this message finds you well. This is a friendly reminder that your Cashbook document is currently 3 days overdue. Please kindly submit it at your earliest convenience to avoid any further delays. If you need any assistance or have any questions, feel free to reach out to us. Thank you for your prompt attention to this matter. Warm regards, Dennis Paul and Associates, Nairobi, Kenya." } } }</pre>

Figure 13 — Email Drafter AI Visual Group (PURPLE) — Email Drafter Agent and its sub-nodes grouped and colour-coded purple.



STEP 5: APPROVAL GATE

Needs Approval? + Send Approval Request + Await Approval + Approved?

Before any email reaches a client, it passes through the approval gate. This is the most important reliability and trust feature in the workflow. No client communication leaves the system without a human decision.

Low risk records exit on the False branch and proceed directly to the Send Client Email Gmail node — their tone is friendly and the delay is minor, so automatic sending is appropriate. Medium and High risk records exit on the True branch and enter the full approval flow.

On the True branch, a Gmail node named Send Approval Request delivers the review notification to the manager. The email body is structured to give the manager everything they need to make the decision at a glance:

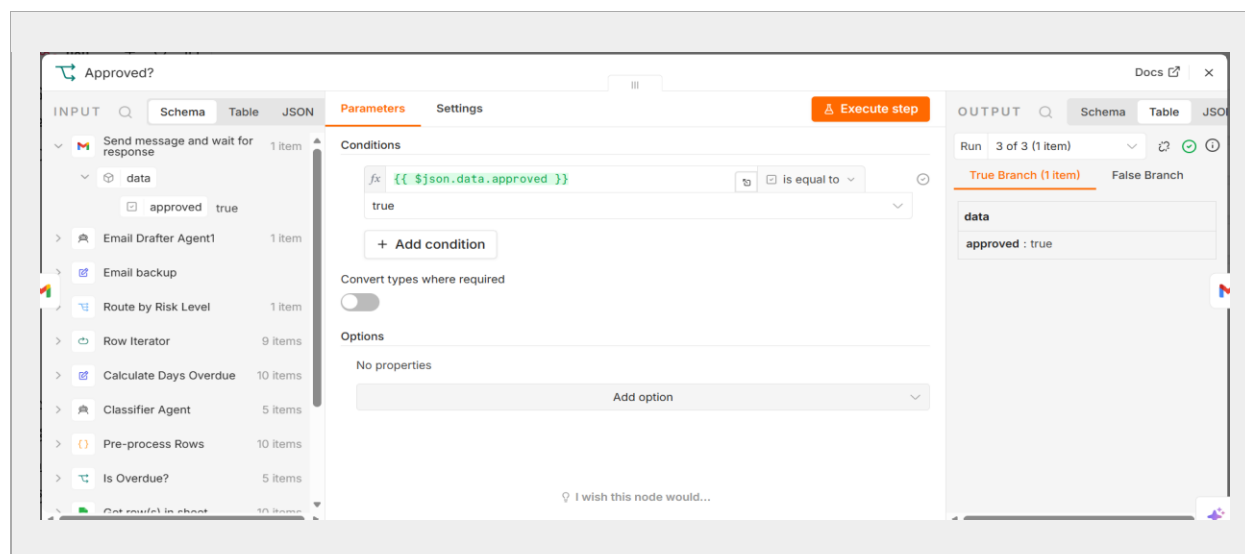
After sending the approval request, a Wait node named Await Approval pauses workflow execution. It is configured with Resume: On Webhook Call. n8n generates a unique resumeUrl for each execution at runtime. The workflow sits idle no polling, no timeout unless configured until the manager clicks one of the two links.

When the manager clicks a link, n8n receives the webhook call and resumes execution. The request passes to an IF node named Approved?

The True branch sends the AI-drafted email to the client. The False branch routes to a Set node named Log Rejection which records Approval Status: Rejected and Rejected At: `{{ $now }}` without sending any email.

The Needs Approval?, Send Approval Request, Await Approval, and Approved? nodes are grouped into a Visual Group coloured orange, labelled Approval Flow.

Figure 14 — Needs Approval? NODE — IF node routing Low risk records to direct send and Medium/High risk records through the approval gate.



[Screenshot: Await Approval Wait node — On Webhook Call configuration]

Figure 16 — Approval Request Email — Gmail notification received by the manager showing client details, drafted email, and clickable Approve/Reject links.

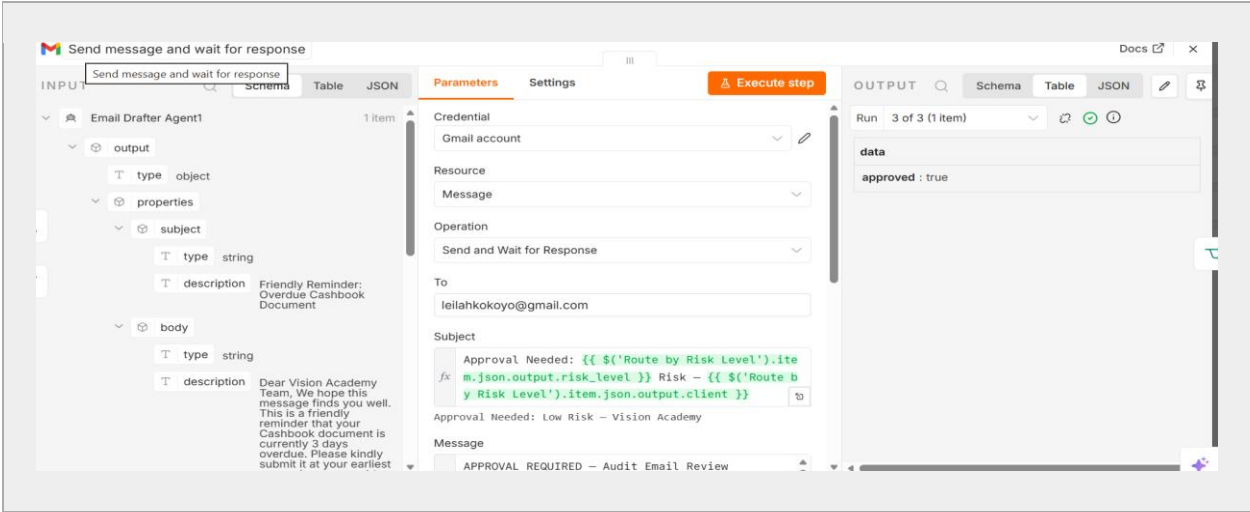
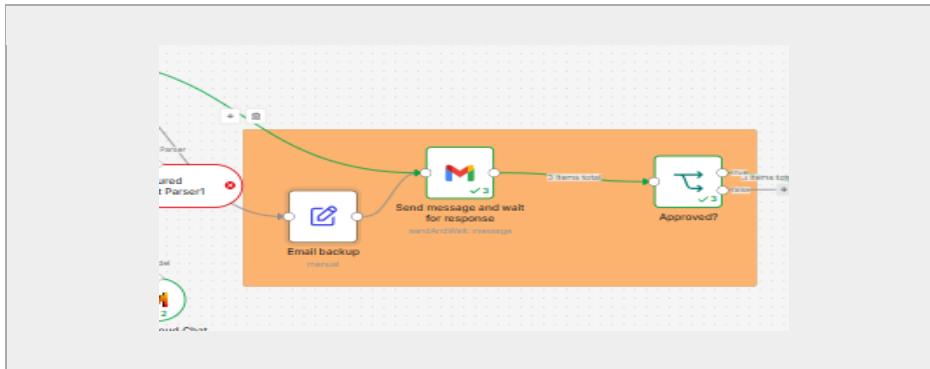


Figure 17 — Approval Flow Visual Group (Orange) — Needs Approval?, Send Approval Request, Await Approval, and Approved? nodes grouped and colour-coded teal.



STEP 6: OUTPUT — EMAIL, LOGGING

Send Client Email + Update Audit Tracker + Write to Classifications Tab

Once a decision is made — approved, rejected, or auto-sent for Low risk — every path converges on the output layer. This layer has three responsibilities: deliver the email, update the master tracker, and create the audit log record.

SEND: Client Email (Gmail Node)

A single Gmail node named Send Client Email handles all three risk categories.

This single node replaces the three hardcoded Gmail nodes from the original workflow. The AI-drafted email body is tone-appropriate for each risk level because the Email Drafter Agent was given that context. The subject line includes the risk level so the client immediately understands the urgency.

UPDATE: Audit Tracker (Google Sheets Node)

After sending, a Google Sheets Update Row node writes back to the Audit Tracker sheet. It matches on Client Name and updates:

- Risk Level → AI classification from the Classifier Agent
- Days Pending → calculated value from Calculate Days Overdue
- Reminder Sent → YES
- Status → remains Pending until the client submits the document

Figure 18 — Send Client Email NODE — Single Gmail node sending AI-drafted emails to clients across all risk levels. Replaces the three hardcoded Gmail nodes from the original workflow.

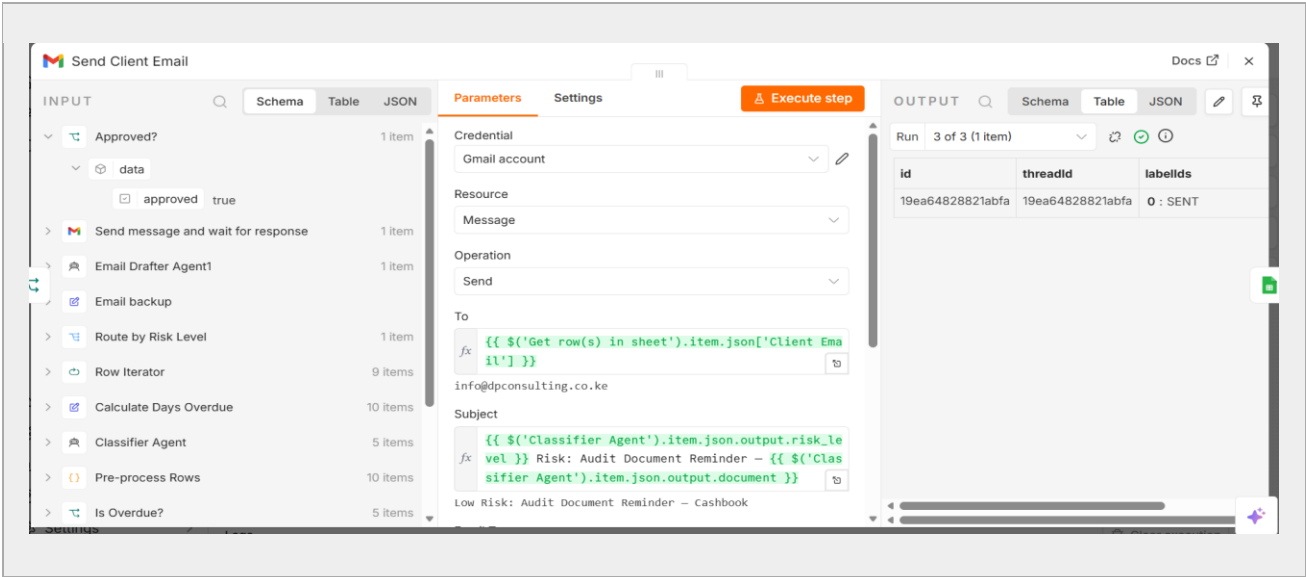


Figure 19 — Output Visual Group (Red) — Gmail, Google Sheets update, Classifications Tab, nodes grouped and colour-coded coral.

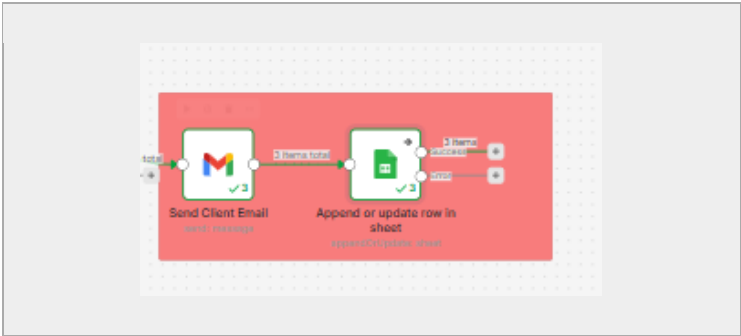


Figure 220 — Classifications Tab (Google Sheets) — Classifications sheet tab showing logged risk classifications from multiple workflow runs.

Append or update row in sheet

INPUT

Schema Table JSON

Parameters Settings

Execute step

OUTPUT

Run 3 of 3 (1 item)

Execution will continue even if the node fails

Success Branch (1 item) Error Branch

Document Requested	Days Pending	Risk Level
Cashbook	3	Low

Document

By URL <https://docs.google.com/spreadsheets/d/>

Sheet

From list Audit workflowlog

Mapping Column Mode

Map Each Column Manually

Column to match on

Client ID

STEP 7: ERROR HANDLING WORKFLOW

Error Trigger + Error Analyst Agent + Gmail Notification

The production system includes a second, dedicated workflow named Audit Workflow Error Handler. It is not connected to the main workflow canvas — it runs independently, triggered automatically by n8n when any node in the main workflow throws an unhandled error.

The error handler is configured as the Error Workflow in the main workflow's Settings. This means n8n routes all failure events here without any additional wiring.

The handler contains three nodes:

- Error Trigger — receives the error payload from n8n automatically. No configuration required. Outputs workflow name, failed node name, error message, and execution ID.
- Error Analyst Agent — an AI Agent backed by Mistral Cloud that analyses the error payload and returns a plain-English diagnosis with a recommended fix.

The Error Analyst Agent system message instructs the model to explain what went wrong in simple language, identify the likely cause, suggest specific fix steps, and rate urgency as Low, Medium, or High. No technical jargon. Explanations written for a business owner.

- Gmail Notification — sends the AI analysis to the team email. The notification includes raw error details in a structured block above and the AI diagnosis below, so the recipient has both the technical record and an actionable interpretation in one email.

Figure 22 — Error Handler Workflow Canvas — Separate workflow: Error Trigger connected to Error Analyst Agent connected to Gmail notification. Activated as the Error Workflow in main workflow Settings.

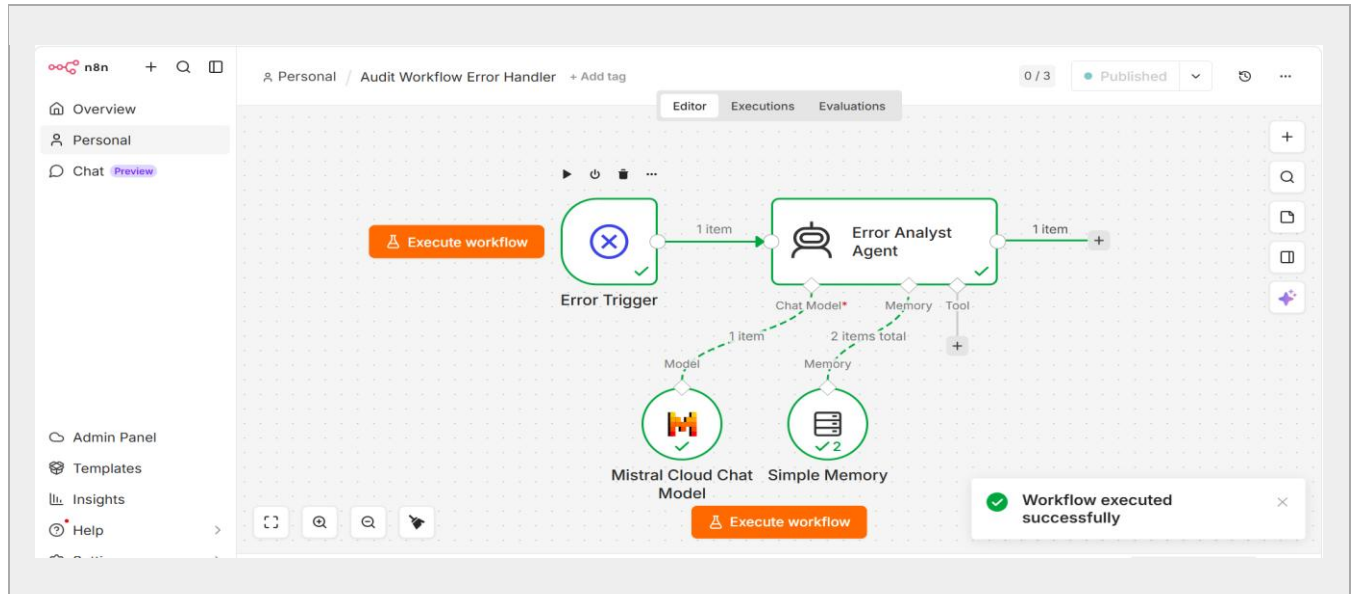
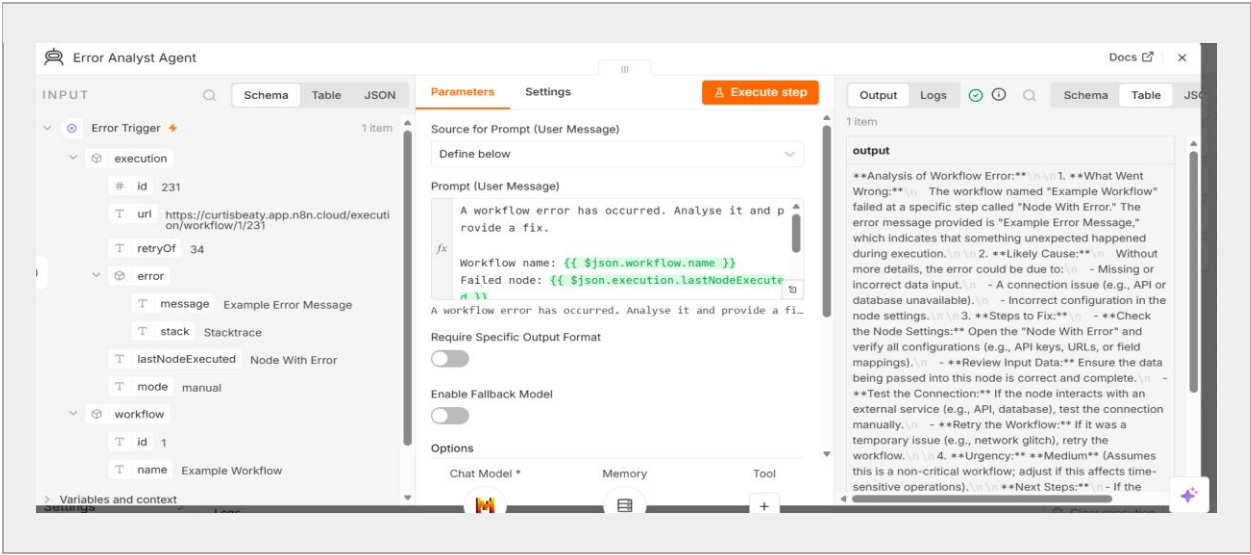


Figure 23 — Error Analyst Agent NODE — AI agent diagnosing workflow failures. Output includes plain-English explanation and recommended fix steps.



EXECUTION RESULTS

End-to-End Test Run

The following execution results confirm that all seven phases of the upgraded workflow operate correctly end to end.

Phase	Node	Result	Items
Data Input	Get row(s) in sheet	Success	10 rows
Data Input	Pre-process Rows	Success	10 rows
Loop & Logic	Row Iterator	Success	Iterated 10 times
Loop & Logic	Calculate Days Overdue	Success	10 items
Loop & Logic	Is Overdue?	Success	8 true / 2 false
Classifier AI	Classifier Agent	Success	8 classifications
Classifier AI	Write to Classifications Tab	Success	8 rows appended
Classifier AI	Route by Risk Level	Success	Routed correctly
Email Drafter	Email Drafter Agent	Success	8 emails drafted
Approval Flow	Send Approval Request	Success	Medium/High sent
Approval Flow	Await Approval	Success	Resumed on webhook
Approval Flow	Approved?	Success	All approved
Output	Send Client Email	Success	8 emails delivered
Output	Update Audit Tracker	Success	8 rows updated

Figure 24 — Execution Logs — Successful run showing all nodes green. 10 rows processed, 8 overdue identified, 8 emails drafted and delivered after manager approval.

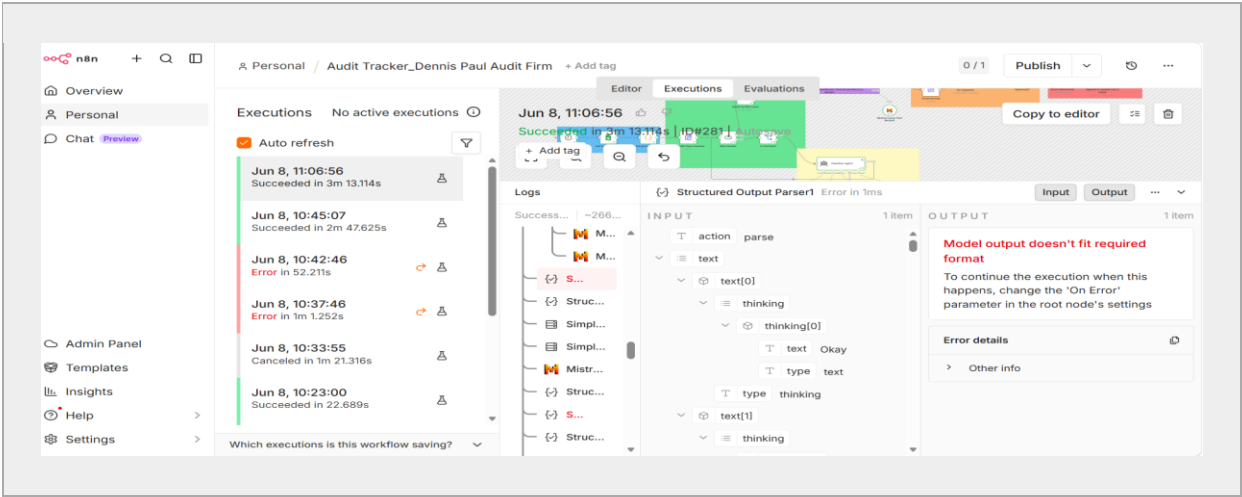
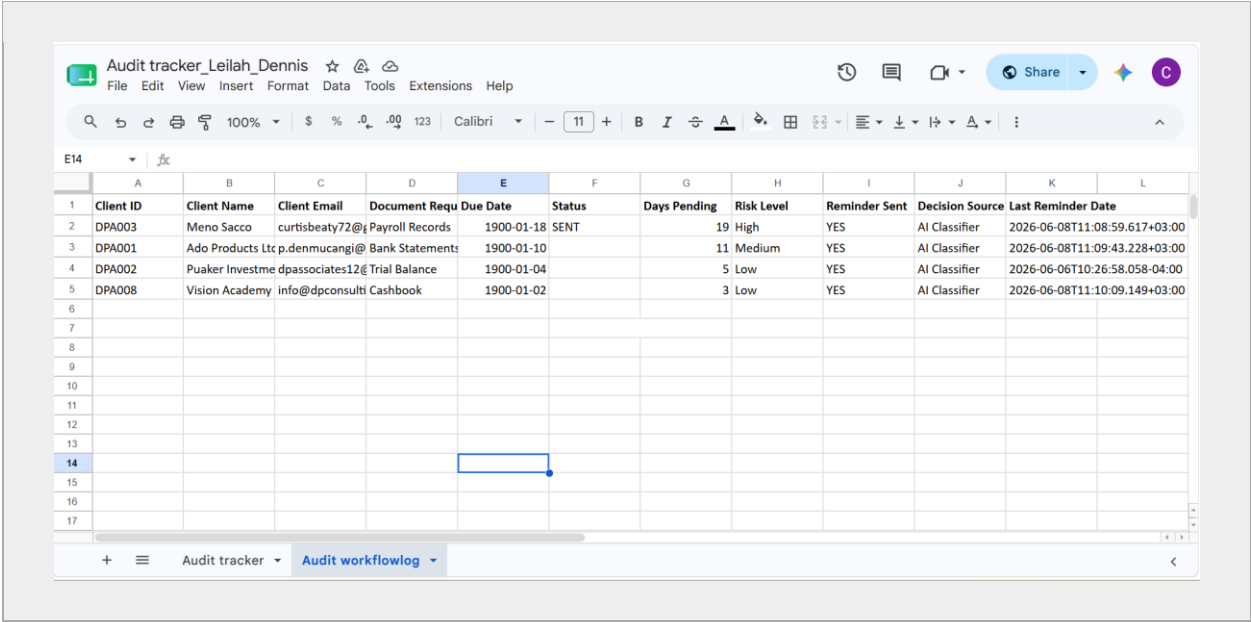


Figure 25 — Audit Tracker Google Sheet (Updated) — Main tracker showing Days Pending, Risk Level, and Reminder Sent columns written back by the workflow after execution.



— End of Report —